VendorFlow: A Unified SaaS Platform for Local Grocery & Pharmacy Digitization

Project Proposal for UCS503P

Submitted to: Dr. Jeelani
Thapar Institute of Engineering and Technology

August 16, 2026

1 Higher-order goal . . . secondary framing

This project aims to strengthen local retail resilience by giving small grocery stores and pharmacies an affordable path to digital commerce. While the broader vision is economic empowerment of neighborhood businesses, the immediate engineering objective is to translate reliable order fulfillment and vendor coordination into a measurable, deployable software solution.

2 Time-to-value . . . primary heuristic

- We will deliver meaningful increments quickly by prototyping the core ordering and vendor-routing workflow and validating it with a small set of pilot vendors within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on vendor and customer feedback.

3 Problem Statement

Small grocery stores and pharmacies in most Indian cities and towns rely on phone calls, WhatsApp, spreadsheets, or paper records to manage orders and inventory. Common pain points include:

- **No online presence:** most vendors cannot afford a custom website, app, or delivery infrastructure of their own.
- **Manual, error-prone operations:** inventory and order tracking done by hand leads to stockouts, missed orders, and delivery mix-ups.
- **Fragmented discovery:** customers have no single place to find nearby stores, compare stock, or track deliveries.
- **Regulatory friction for pharmacies:** prescription-backed orders need a verification step that ad-hoc channels (calls, chat apps) do not support reliably.

Why this matters now (contextual relevance): Consumer demand for online ordering continues to grow, but the smallest vendors are the ones least able to build this infrastructure themselves; a shared platform lets many small businesses digitize at once without each bearing the full engineering cost.

4 Proposed Solution . . . Software Proposition

4.1 Overview

Build a full-stack SaaS platform, **VendorFlow**, with the following components:

- **Customer portal:** discover nearby grocery/pharmacy vendors, browse products, upload prescriptions where required, place orders, and track deliveries.

- **Vendor portal:** onboarding and approval workflow, inventory management, order management, and a reliability/analytics dashboard.
- **Delivery partner portal:** accept assigned deliveries and update status in real time.
- **Admin portal:** vendor approval, prescription verification oversight, platform analytics, and audit logs.
- **Smart vendor routing:** route an order to the best-fit vendor using distance, live stock, and a computed reliability score.
- **Credit system:** an in-platform credit/wallet mechanism for customers and vendors, supporting promotional credits, refunds, and settlement between platform and vendors.

4.2 Core workflow

1. Customer browses nearby vendors and adds items to cart (uploading a prescription for pharmacy items where required).
2. System applies smart vendor routing (distance + stock + reliability score) and, for pharmacy orders, queues the prescription for manual verification.
3. Order is confirmed, a delivery partner is assigned, and status updates flow through the order lifecycle (Placed $\rightarrow$ Confirmed $\rightarrow$ Out for Delivery $\rightarrow$ Delivered).
4. Customer and vendor receive real-time notifications at each stage; credits/wallet balances are updated on completion, refund, or promotional events.

4.3 Operational constraints for an educational, production-style setting

- Keep the customer- and vendor-facing UIs usable on low-to-mid range devices via responsive web design.
- Keep the order-matching and routing logic heuristic and explainable rather than research-grade; focus on correctness, latency, and auditability.
- Design the backend for high throughput, since order placement, routing, and notification delivery are on the platform's critical path.

5 Solution Approach . . . Engineering focus

This is an engineering course project, so we focus on system architecture, performance, and correctness of the transactional workflow rather than novel algorithm research.

5.1 Performance-focused backend . . . non-research

The core transactional services (order placement, vendor routing, inventory updates) are engineering-heavy and latency-sensitive, so the approach is architectural rather than algorithmic:

- Implement the high-throughput backend services (order intake, routing engine, inventory locking) in **C++** for predictable low-latency performance under load.
- Use **Python** for auxiliary services: analytics jobs, reliability-score recomputation, and admin/reporting tooling.
- Use **JavaScript** for the customer, vendor, delivery, and admin front-ends, and for lightweight orchestration/API-gateway glue code where appropriate.
- Vendor routing itself remains a simple weighted-score heuristic (distance + stock availability + reliability score) without claiming state-of-the-art optimization.

5.2 System architecture . . . web-based solution

A multi-tier architecture:

- **Frontend:** responsive web apps for the four portals (customer, vendor, delivery partner, admin), built in JavaScript.
- **High-performance backend (C++):** REST/WebSocket services for authentication, order lifecycle, vendor routing, inventory locking, and the credit/wallet ledger.
- **Auxiliary services (Python):** background jobs (reliability score updates, routing-timeout handling, notification queue processing) and analytics pipelines.
- **Cache/queue layer: Redis** for caching, rate limiting, session data, and background job/notification queues.
- **Primary database: PostgreSQL** for users, vendors, products, inventory, orders, credit/wallet ledger, reviews, and audit logs.
- **Real-time layer:** Socket.IO (or an equivalent WebSocket layer) for real-time order and delivery status notifications.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests (C++ unit tests, Python service tests, frontend tests) on every change.
- Produce repeatable deployment artifacts to staging for each service.
- Enable safe and frequent releases of incremental features across portals.

6 Evaluation Criterion . . . measurable and attributable

We define success using measurable metrics tied to the core order-fulfillment workflow.

6.1 Primary evaluation metric

Order-to-confirmation latency (OCL): time between order placement and vendor confirmation (post-routing) in the system.

- Target: median OCL ≤ 60 seconds during pilot window.
- Attribution: measured from backend timestamps (order-placed event vs. vendor-confirmed event).

6.2 Secondary evaluation metrics

- **Delivery success rate:** percentage of orders marked Delivered without escalation or cancellation.
- **Vendor reliability accuracy:** correlation between computed reliability score and actual on-time fulfillment during the pilot.
- **Prescription verification turnaround:** median time from upload to admin-verified status for pharmacy orders.
- **System reliability:** availability of staging/production for login, ordering, and routing during business hours (e.g., $\geq 99\%$ uptime in pilot).
- **Backend throughput:** sustained requests/second the C++ order and routing services can serve under load-test conditions.

6.3 Pilot validation plan . . . high-level

- Recruit 10–20 pilot customers and 3–5 vendor accounts (grocery and pharmacy mix, simulated where real vendors are unavailable).
- Compare metrics before/after within the iteration window, using short interviews or existing process observations to estimate baseline manual-process timings.

7 Scalability . . . with foresight

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in vendors, orders, and concurrent portal usage by:
   - decoupling the C++ backend, Python auxiliary services, and frontends,
   - enabling pagination, indexing, and read replicas for common PostgreSQL queries,
   - using Redis for caching and rate limiting to shield the primary database,
   - designing backend APIs to be stateless where possible.
2. **Operationally deployable (practically):** The system should run on common infrastructure:
   - managed PostgreSQL and Redis instances,
   - containerized deployment for the C++ backend, Python services, and frontends,
   - CI/CD pipeline for staging/production across all services.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a production-style project:

- High-performance C++ web/service framework for the core backend.
- Python framework for auxiliary/background services and analytics.
- JavaScript framework for the four frontend portals.
- PostgreSQL as the managed relational database.
- Redis for caching, rate limiting, and job/notification queues.
- Authentication approach (JWT-based, with role-based access control).
- Object storage for prescription images and product photos.
- Test frameworks for unit/integration tests across C++, Python, and JavaScript components.
- CI runner and staging deployment target.

We will use these mature components to focus on workflow design, backend performance, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables . . . iteration-friendly

9.1 Initial deliverable . . . first iteration

- Customer, Vendor, Delivery Partner, and Admin portals with JWT authentication and role-based access control.
- Vendor onboarding & approval workflow; product & inventory management.
- Smart vendor routing (distance + stock + reliability) implemented in the C++ backend.

- Shopping cart, checkout, and order lifecycle management.
- Prescription upload with manual admin verification.
- Delivery assignment and status-based tracking.
- Basic logging and timestamps for OCL measurement.
- CI: build + backend (C++/Python) unit tests + linting.
- CD to staging with a single pipeline job.

9.2 Subsequent deliverables

- Credit/wallet system for customers and vendors (promotional credits, refunds, settlement).
- Real-time notifications via Socket.IO, plus email and in-app notifications.
- Reviews & ratings, feeding into the vendor reliability score.
- Vendor & admin analytics dashboards.
- Redis-backed caching and rate limiting hardened for scale.
- Background jobs for notification queueing, routing timeouts, and reliability-score recomputation.
- Audit logs and activity history across vendor and admin actions.
- Improved search/filtering and indexed queries for scale readiness.

10 Risks and mitigations

- **Data privacy / consent:** minimize collected data, especially prescription images; store only what is needed and restrict access to verified admins.
- **Vendor adoption:** keep the vendor portal simple, with minimal steps for onboarding, inventory updates, and order confirmation.
- **Backend complexity (C++):** mitigate with strong test coverage, clear service boundaries, and CI-enforced builds before merge.
- **Evaluation measurement ambiguity:** instrument timestamps and define clear order-lifecycle states before development begins.
- **Operational issues in pilot:** use staging early; ensure CI/CD produces repeatable deployments for all services.

11 Summary

This project proposes VendorFlow, a deployable SaaS platform that helps small grocery stores and pharmacies digitize inventory, ordering, delivery, and vendor management, with a high-performance C++ backend, Python auxiliary services, PostgreSQL and Redis for storage and caching, and JavaScript frontends across four portals. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially order-to-confirmation latency). It remains focused on engineering workflow, backend performance, and scalability readiness rather than research-driven novelty.